

NAME: S.ROHINI
 REG.NO: 192424413
 CODE: CSA0406
 COURSE: OPERATING SYSTEMS
 DATE: 03/08/2026

Q1. Deadlock Debugging:

Problem Identification:

The deadlock occurs because both processes lock the resources in different orders.

- * Process P1 locks R1 first and then requests R2.
- * Process P2 locks R2 first and then requests R1.
- * If both processes execute simultaneously, P1 waits for R2 while P2 waits for R1.
- * This creates a Circular Wait, causing a deadlock.

Debugging:

Execution Flow:

P1 locks R1

↓

P2 locks R2

↓

P1 waits for R2

↓

P2 waits for R1

↓

Deadlock

Optimization:

Use a fixed resource allocation order.

lock(R1)

lock(R2)

Critical Section

unlock(R2)

unlock(R1)

Both processes should follow the same order.

Another optimization is to use trylock(). If the second resource is unavailable, release the first resource and retry later.

Analysis:

Using the same locking order removes the Circular Wait condition and completely prevents deadlock while maintaining the original process logic.

Q2. CPU Utilization Problem:

Problem Identification:

The system shows only 30% CPU utilization, even though processes are available.

The scheduler log shows:

Ready → Waiting → Ready → Waiting → Ready

This indicates that processes are spending most of their time waiting for I/O operations instead of using the CPU. As a result, the CPU remains idle frequently.

Debugging:

Root Cause:

- * Processes are I/O-bound.
- * Frequent I/O requests move processes from the Ready state to the Waiting state.
- * When processes wait for I/O, the CPU has no process to execute, resulting in low CPU utilization.

Execution Flow:

Process Ready

↓

CPU Executes

↓

I/O Request

↓

Waiting State

↓

I/O Complete

↓

Ready State

Optimization:

Optimization 1: Increase Multiprogramming

- * Keep more processes in the Ready Queue.
- * While one process waits for I/O, another process can use the CPU.
- * This improves CPU utilization.

Optimization 2: Use Asynchronous I/O

- * Perform I/O operations without blocking the CPU.
- * The CPU continues executing other processes while I/O is in progress.
- * This reduces CPU idle time and increases system throughput.

Analysis

By increasing multiprogramming and using asynchronous I/O, the CPU remains busy even when some processes are waiting for I/O. These optimizations improve CPU utilization, reduce idle time, and increase overall system performance.

Q3. Memory Bottleneck Debugging**Problem Identification:**

The system experiences frequent page faults, which slow down the performance of the multi-threaded program.

Memory Map:

- * Code = 5 MB
- * Heap = 150 MB
- * Stack = 1 MB per thread
- * Threads = 25
- * Physical RAM = 2 GB

The main issue is that multiple threads and a large heap increase memory usage, causing frequent page faults and reducing performance.

Debugging:**Root Cause:**

- * Large heap memory consumes more RAM.
- * Each of the 25 threads requires its own stack memory.
- * Frequent access to different memory pages causes page faults.
- * The CPU spends more time loading pages from memory instead of executing the program.

Memory Usage:

Code = 5 MB
 Heap = 150 MB
 Thread Stack= 25 × 1 MB = 25 MB

 Total Used = 180 MB

Although the total memory is within 2 GB RAM, inefficient memory access leads to excessive page faults and slower execution.

Optimization:**Optimization 1: Reduce Heap Memory Usage**

- * Remove unused objects.
- * Reuse objects whenever possible.
- * Free memory after use to reduce heap size.

Optimization 2: Use a Thread Pool

- * Reduce the number of active threads.
- * Reuse existing threads instead of creating new ones.
- * This decreases stack memory usage and improves memory efficiency.

Analysis:

Reducing heap memory usage and using a thread pool decreases page faults, improves memory utilization, and increases overall system performance. These optimizations reduce execution time and enhance the efficiency of the multi-threaded application.

Q4. Starvation Debugging**Problem Identification:**

In the Multilevel Queue Scheduler, process P5 is placed in Q3 (Batch Queue).

Queue Structure:

- * Q1 - Interactive (Highest Priority)
- * Q2 - System
- * Q3 - Batch (FCFS)

Since Q1 always has the highest priority, new interactive processes continuously execute before Q3. As a result, P5 never gets CPU time, causing starvation.

Debugging:**Root Cause:**

- * Q1 processes always execute before Q2 and Q3.
- * New interactive processes keep arriving in Q1.
- * Q3 processes continue waiting indefinitely.
- * Therefore, P5 remains in the Ready Queue without execution.

Execution Flow:

Q1 Process Arrives



CPU Executes Q1



Another Q1 Process Arrives



Q3 (P5) Keeps Waiting



Starvation

Optimization:

Optimization 1: Apply Aging:

- * Increase the priority of P5 based on its waiting time.
- * After waiting for a specific period, move P5 from Q3 to Q2 or Q1.
- * This ensures every process eventually gets CPU time.

Optimization 2: Allocate CPU Time to Q3:

- * Reserve a fixed percentage of CPU time for the Batch Queue.
- * This guarantees that Q3 processes are executed even when higher-priority queues are busy.

Analysis:

Applying Aging and allocating CPU time to lower-priority queues prevents starvation while maintaining the responsiveness of higher-priority processes. These optimizations improve fairness, ensure all processes are executed, and enhance overall scheduling performance.

Q5. Context-Switch Overhead Debugging

Problem Identification:

The server performs 20,000 context switches per second, resulting in high CPU overhead.

System Details:

- > Average burst time = 2 ms
- > I/O wait frequency = Very High
- > Scheduler = Preemptive
- > Time Quantum = 1 ms

The main issue is that the time quantum is too small (1 ms). Processes are preempted frequently before completing execution, causing excessive context switching.

Debugging:

Root Cause:

- * Each process has an average burst time of 2 ms, but the scheduler allows only 1 ms of execution.
- * Processes require multiple context switches to complete.
- * Frequent I/O operations further increase the number of context switches.
- * The CPU spends more time switching between processes than executing them.

Execution Flow:

Process Starts



Runs for 1 ms



Context Switch



Runs Again



I/O Wait



Context Switch

Optimization:

Optimization 1: Increase Time Quantum:

- * Increase the time quantum from 1 ms to 4-5 ms.
- * This allows processes to complete with fewer context switches.
- * CPU overhead is reduced.

Optimization 2: Optimize I/O Operations:

- * Use asynchronous I/O or I/O buffering.
- * Reduce unnecessary blocking during I/O operations.
- * This decreases the frequency of context switches and improves CPU efficiency.

Analysis:

Increasing the time quantum reduces unnecessary preemption, while optimizing I/O operations minimizes process blocking. Together, these optimizations lower context-switch overhead, improve CPU utilization, increase throughput, and enhance overall system performance.